

SQL Query Explanations - Pizza Sales Analysis

KPI QUERIES EXPLAINED

Query 1: Total Revenue

```
SELECT SUM(total_price) AS Total_Revenue FROM pizza_sales;
```

- SUM(total_price): Adds up the total price of all pizzas sold.
- AS Total_Revenue: Alias for readability.
- Purpose: Get the total revenue from pizza sales.

Query 2: Total Orders

```
SELECT COUNT(DISTINCT order_id) AS Total_Orders FROM pizza_sales;
```

- COUNT(DISTINCT order_id): Counts unique orders.
- DISTINCT avoids duplicates from multi-pizza orders.
- Purpose: Total number of individual orders placed.

Query 3: Total Pizzas Sold

```
SELECT SUM(quantity) AS Total_Pizzas FROM pizza_sales;
```

- SUM(quantity): Adds up all quantities of pizzas sold.
- Purpose: Total number of pizzas sold.

Query 4: Average Order Value

```
SELECT SUM(total_price) / COUNT(DISTINCT order_id) AS Avg_Order_Value FROM pizza_sales;
```

- SUM(total_price): Total revenue.
- COUNT(DISTINCT order_id): Unique orders.
- Division gives average spent per order.

Query 5: Average Pizzas Per Order

```
SELECT ROUND(SUM(quantity) / COUNT(DISTINCT order_id), 2) AS Avg_Pizza_Per_Order FROM pizza_sales;
```

- SUM(quantity): Total pizzas sold.
- COUNT(DISTINCT order_id): Total orders.
- ROUND(, 2): Rounds to 2 decimal places.

CHART-RELATED QUERIES EXPLAINED

Daily Trend for Total Orders

```
SELECT DATENAME(WEEKDAY, order_date) AS Day_Name, COUNT(DISTINCT order_id) AS  
No_of_orders FROM pizza_sales GROUP BY DATENAME(WEEKDAY, order_date);
```

- DATENAME(WEEKDAY, order_date): Extracts day of the week.
- COUNT(DISTINCT order_id): Orders per day.
- GROUP BY groups data per weekday.

Monthly Trend for Total Orders

```
SELECT DATENAME(MONTH, order_date) AS Month_Name, COUNT(DISTINCT order_id) AS  
No_Of_Orders FROM pizza_sales GROUP BY DATENAME(MONTH, order_date);
```

- DATENAME(MONTH, order_date): Extracts month name.
- COUNT(DISTINCT order_id): Orders per month.
- GROUP BY groups data by month.

Percentage of Sales by Pizza Category

```
SELECT pizza_category, ROUND(SUM(total_price), 2) AS Sales, ROUND(SUM(total_price)*100/(SELECT  
SUM(total_price) FROM pizza_sales), 2) AS percent_sales FROM pizza_sales GROUP BY pizza_category;
```

- SUM(total_price): Revenue per category.
- ROUND(): Percentage share of each category.

Percentage of Sales by Pizza Size

```
SELECT pizza_size, ROUND(SUM(total_price), 2) AS Sales, ROUND(SUM(total_price)*100/(SELECT  
SUM(total_price) FROM pizza_sales), 2) AS percent_sales FROM pizza_sales GROUP BY pizza_size;
```

- Same logic as pizza category but by size.

Total Pizzas Sold by Category

```
SELECT pizza_category, SUM(quantity) AS pizzas_sold FROM pizza_sales GROUP BY pizza_category;
```

- SUM(quantity): Number of pizzas sold by category.

Top 5 Best Sellers by Revenue

```
SELECT TOP 5 pizza_name, SUM(total_price) AS Revenue FROM pizza_sales GROUP BY pizza_name
```

ORDER BY Revenue DESC;

- Identifies top-selling pizzas by revenue.

Top 5 Best Sellers by Quantity

```
SELECT TOP 5 pizza_name, SUM(quantity) AS Quantity FROM pizza_sales GROUP BY pizza_name  
ORDER BY Quantity DESC;
```

- Shows best sellers by quantity.

Top 5 Best Sellers by Total Orders

```
SELECT TOP 5 pizza_name, COUNT(DISTINCT order_id) AS Total_Orders FROM pizza_sales GROUP BY  
pizza_name ORDER BY Total_Orders DESC;
```

- Most frequently ordered pizzas.

Bottom 5 Worst Sellers by Revenue/Quantity/Orders

- Same as top sellers but use ORDER BY ASC.

Common SQL Functions Used:

- SUM(): Adds numeric values.
- COUNT(): Counts rows.
- DISTINCT: Avoids duplicates.
- ROUND(): Rounds numeric results.
- GROUP BY: Aggregates by a column.
- ORDER BY: Sorts data.
- TOP N: Limits results.
- DATENAME(): Extracts part of date.
- Arithmetic (*100): Used for percentage calculations.